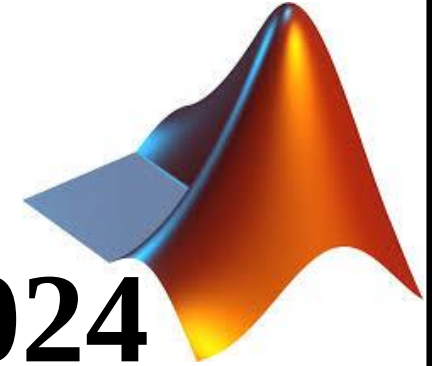


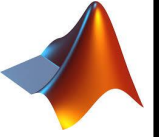


MathWorks Modeling Award 2024 Submission

- ***Faculty mentor name:*** Dr. Sanjay B. Chikalthankar
- ***Institution name:*** Government College of Engineering, Chhatrapati Sambhajinagar
- ***Team Name:*** Team GECCS
- ***Team Leader Name:*** Sushant Khamaitkar
- ***E-mail address of the team leader:*** skhamaitkar98@gmail.com
- ***Mobile number of the team leader:*** +91 8888082538

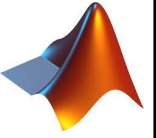
TEAM CYBROTICS

MathWorks Toolboxes and Support packages used:

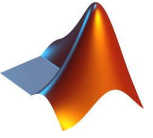


- ☐ Simscape Multibody
- ☐ Control Design toolbox
- ☐ Robotic System toolbox
- ☐ MATLAB coder and Simulink coder
- ☐ MATLAB GUI (guide) Computer vision toolbox
- ☐ MATLAB support package for Arduino
- ☐ Simulink support package for Arduino
- ☐ MATLAB support package for USB Webcams
- ☐ Image Acquisition Toolbox
- ☐ Image Processing Toolbox
- ☐ State flow
- ☐ Data Acquisition toolbox
- ☐ Mobile Robotics Simulation Toolbox

Application in overall development process:

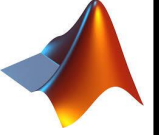


- We utilized **Simscape Multibody** to simulate and analyze the dynamic behavior of complex mechanical systems in Robot 1 and Robot 2, empowering us to design, optimize, and control them more effectively.
- We used **Stateflow** in MATLAB and Simulink for navigation design, employing finite state machines (FSMs) for efficient movement planning. Alongside, the **Robotics System Toolbox** offered a range of tools for tasks like kinematics, path planning, and sensor fusion, enhancing our robot's capabilities. We used MATLAB's Mobile Robotics Simulation Toolbox to refine and test navigation algorithms for our robot in simulated environments before deploying them onto the physical robot.
- Implemented PID controllers for robots using MATLAB's **Control Design Toolbox**, refining and optimizing tailored control models in MATLAB and Simulink to ensure precise and stable performance in tasks such as position control, and stabilization.



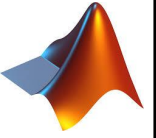
- Implemented **MATLAB support package for USB webcams** in our robot to capture images, utilized **Image Acquisition and Processing Toolboxes** for object detection, enabling visual data analysis for behavior design of Robot 2.
- Integrated **Data Acquisition Toolbox** in MATLAB and Simulink for seamless connection with sensors and instruments in our robot, enabling real-time data acquisition, signal processing, and visualization within the MATLAB environment.
- The **MATLAB Support Package for Arduino** and **Simulink Support Package for Arduino** enable the design, simulation, and deployment of algorithms developed in MATLAB and Simulink onto Arduino-based embedded systems, facilitating rapid prototyping and implementation of control, sensing, and communication functionalities in various applications.
- We used **MATLAB Coder** and **Simulink Coder** to automatically generate C/C++ code from MATLAB scripts and Simulink models, enhancing our robot's performance with direct deployment onto embedded hardware for real-time execution.

Programs done using MATLAB:



- ☐ Modelling and Simulation of Dynamic behaviour of Robot 1 and Robot 2.
- ☐ Analysis of locomotion and the algorithms for Robot 1 and Robot 2 .
- ☐ Designing the control system.
- ☐ Calculations of physical and mechanical parameters of the model.
- ☐ Calculating trajectory of Ball with different angles and velocities.
- ☐ Designing the algorithms using state flow.
- ☐ Data analysis for fast execution of algorithms.
- ☐ Capturing co-ordinates and path follow using Mobile Robotics Simulation Toolbox.
- ☐ Serial Communication between Arduino and MATLAB.

Overall development process followed :



- ☐ Requirements of latest versions, different toolbox, support packages of MATLAB.
- ☐ System Integration and Calibration using Simulink.
- ☐ Software integration according to basic needs.
- ☐ Different seedling picking and placing, ball grabbing and transferring mechanisms design and analysis.
- ☐ Data analysis of Gripping force, Centrifugal forces, max height of trajectory, Velocity, driving force of transfer and Grabbing Mechanism
- ☐ Analysis and implementation of LSA08(Advanced Auto-Calibrating Line Sensor) on Robot 2 using Mobile Robotics Simulation Toolbox.
- ☐ Design and development in Robot 1 and Robot 2's locomotion using PID controlled Tuning and Data analysis.
- ☐ Code generation and model encryption.
- ☐ Analysis of image processing.
- ☐ Detailed design for faster prototyping.
- ☐ Using scripts in Simulink models for easy handling and faster understanding.

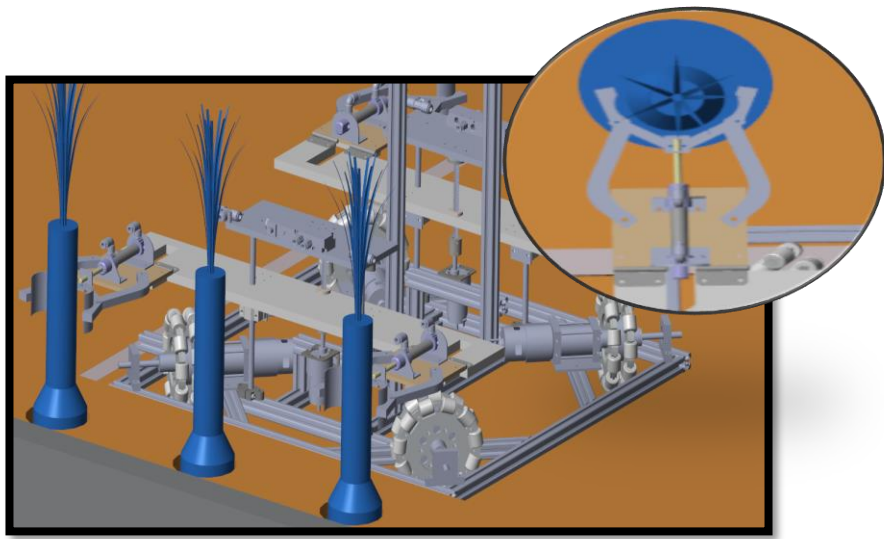


Fig.1(a) R1 Picking seedlings

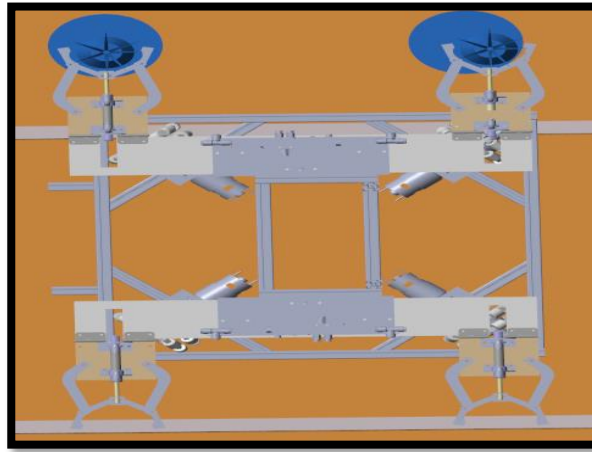


Fig.1(b) Placing seedlings

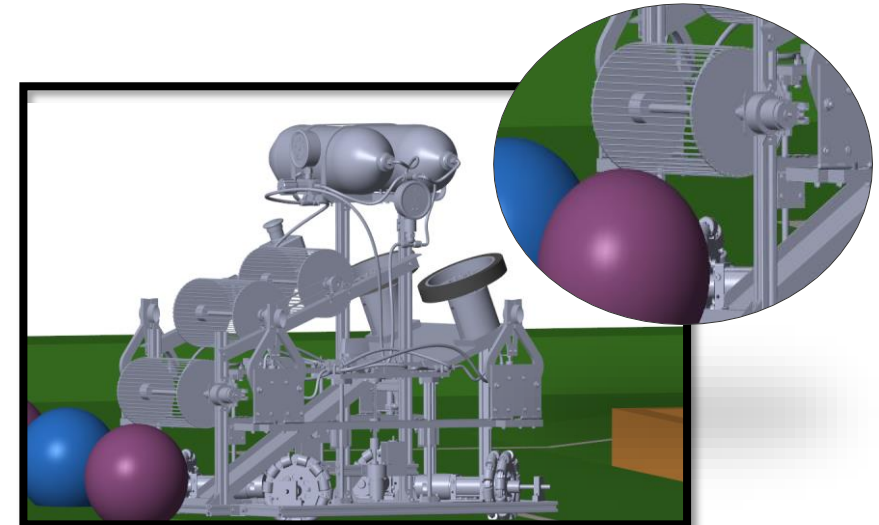


Fig.1(c) Grabbing ball

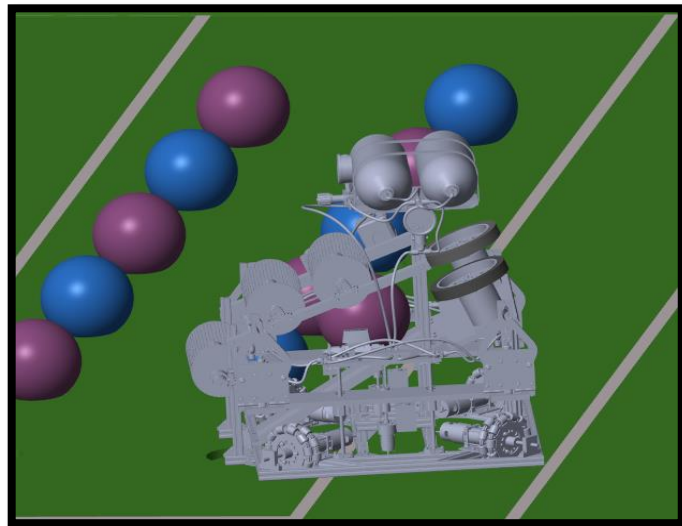


Fig.1(d) Ball feeding to transfer mechanism

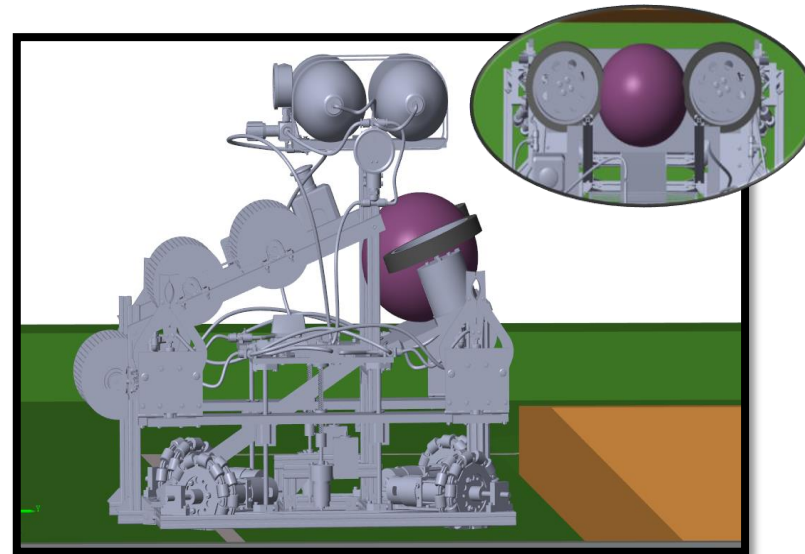


Fig.1(e) Compressed ball between two counter rotating wheels

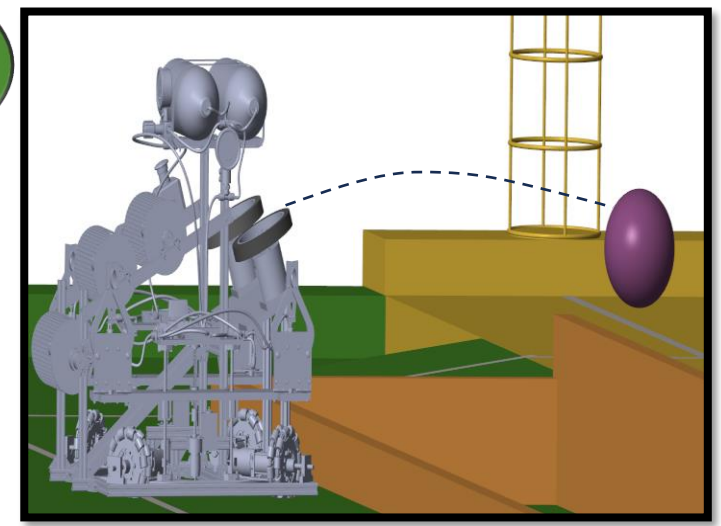


Fig.1(f) After releasing ball to storage zone(Ball trajectory)

Fig.1. Process flow of tasks of Robot 1

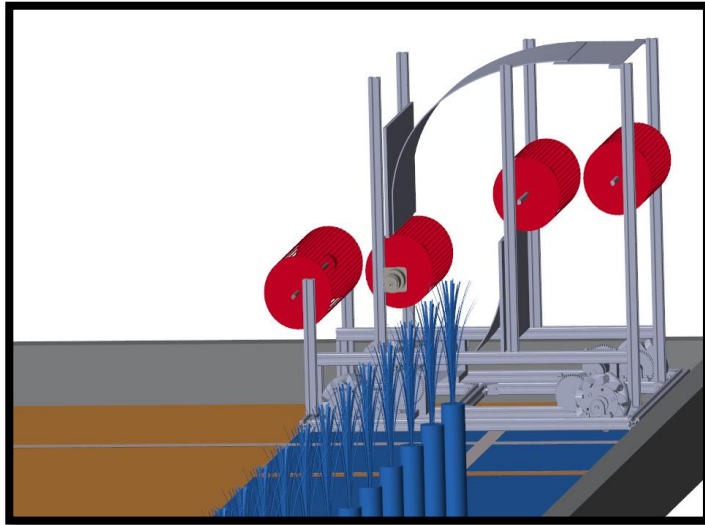


Fig.2(a) At Start Zone

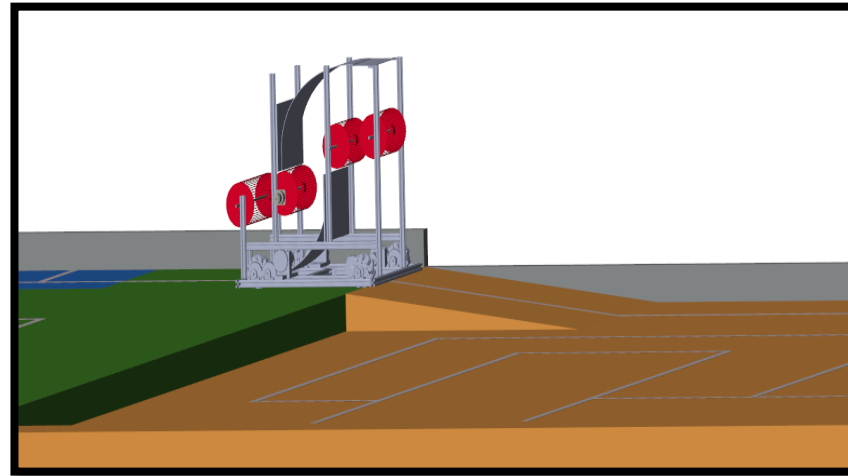


Fig.2(b) Moving towards Area 2

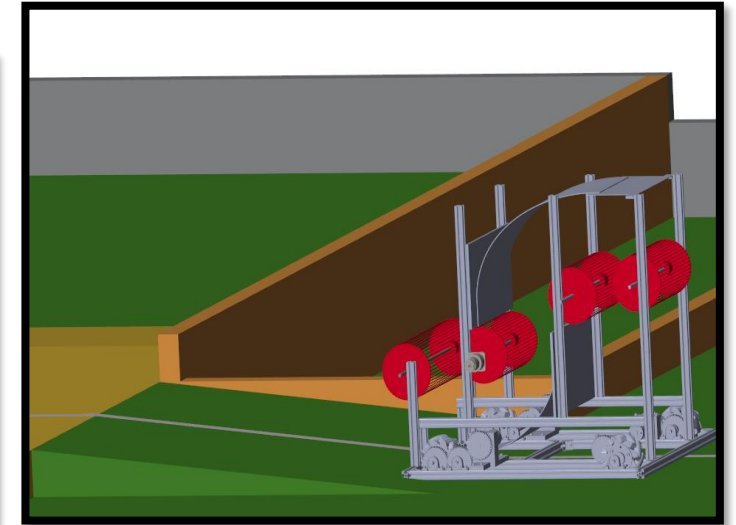


Fig.2(c) Moving towards Area 3

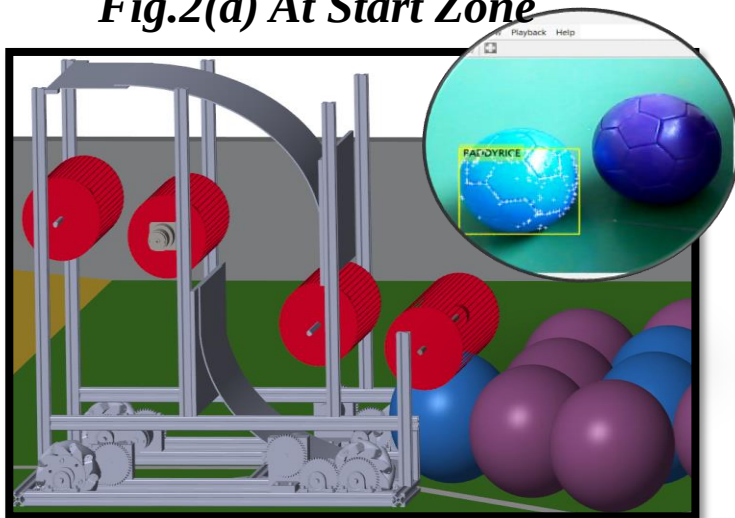


Fig.2(d) Detecting desired ball and grabbing it

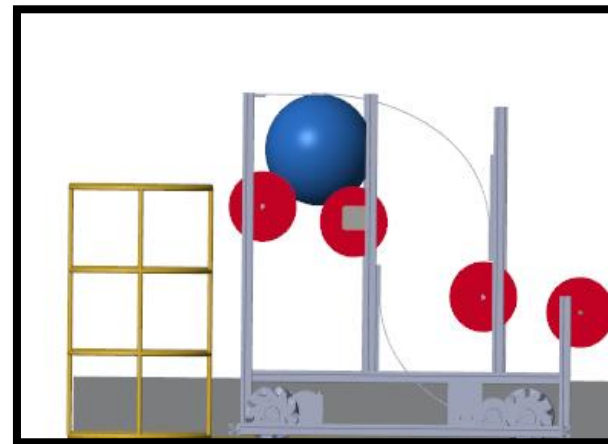


Fig.2(e) Storing ball into Silo

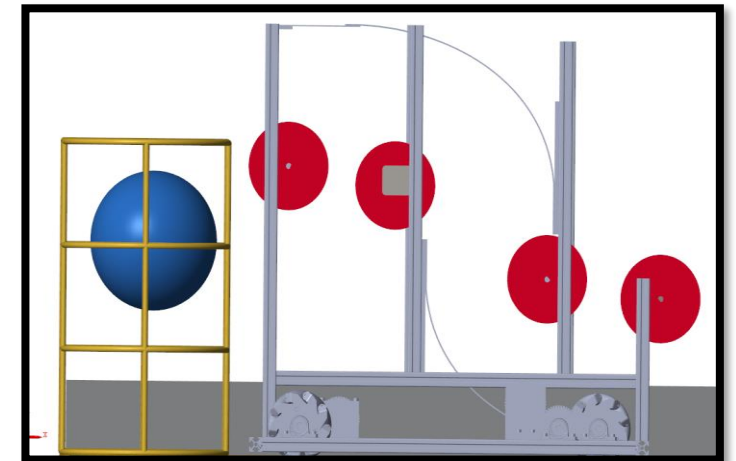


Fig.2(f) Ball being stored into Silo

Fig.2. Process flow of tasks of Robot 2

Plant and Robot Modelling:

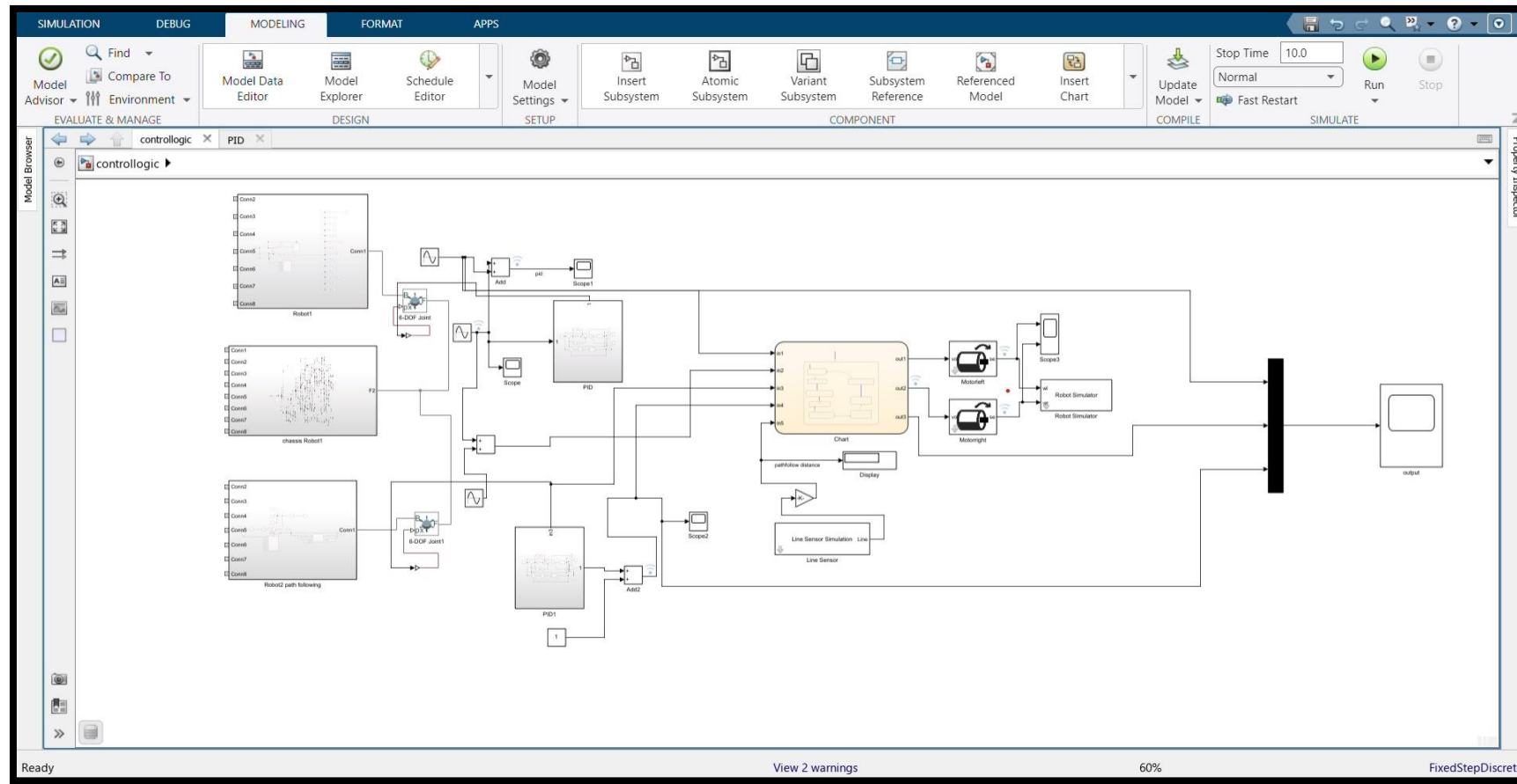
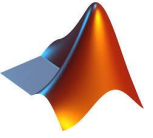
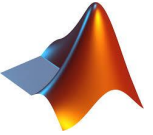


Fig.3. Plant model

- The plant model for the overall system is build using the blocks from previously mentioned mathworks library.



Modelling and Simulation of Dynamic Behavior of Robot 1 and Robot 2 using MATLAB, Simulink and Simscape Multibody:

CAD Model Creation and Import:

- The Robot 1 and Robot 2 CAD models were created in SolidWorks.
- These models were converted to “XML” files.
- The “XML” files were imported into MATLAB using the “smimport” command from Simscape Multibody.

Dynamic Simulations:

- The integration captures the physical characteristics and interactions of the robots and their components.
- This allows for the creation of dynamic simulations.

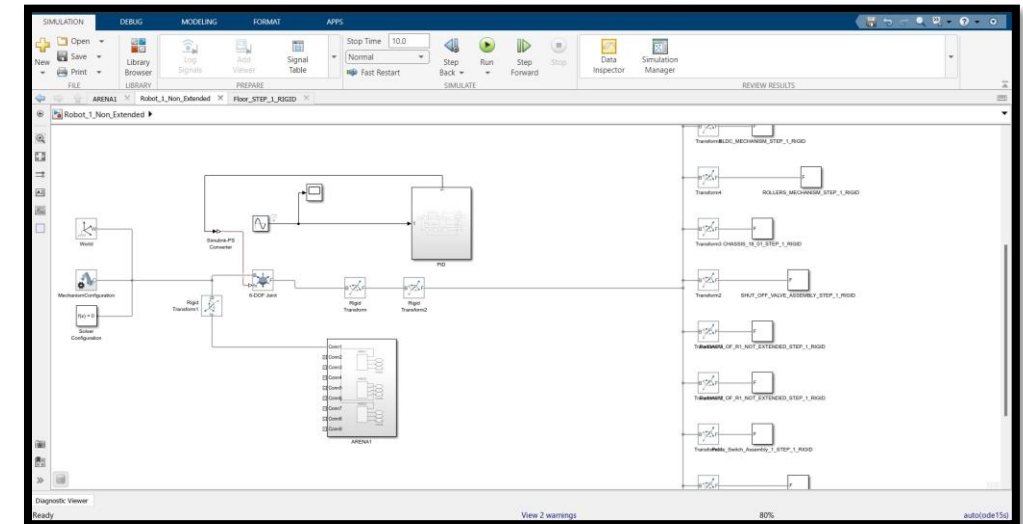


Fig.4. Simscape Multibody of Robot 1

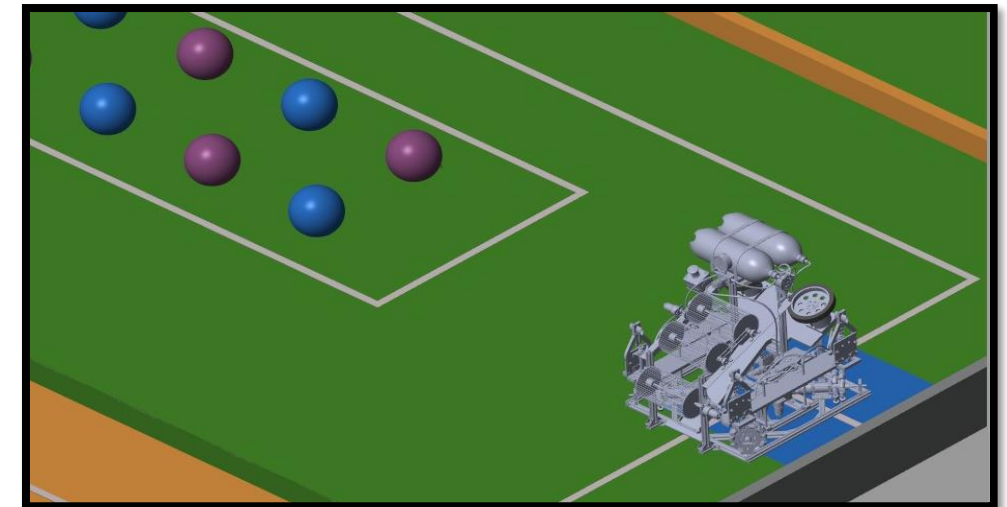


Fig.5. Robot 1 in Area 2

Physical Parameters Calculation:

- Gripping force of the pneumatic gripper.
- Ball trajectory.
- Speed of motors for the chassis and rollers.

Model-Based Design:

- Exploration of different mechanisms using model-based design.
- Reduction in production cost and development time.

2D Kinematic Models:

- Development of 2D kinematic models for robot geometries such as differential chassis drive and four-wheeled robots.

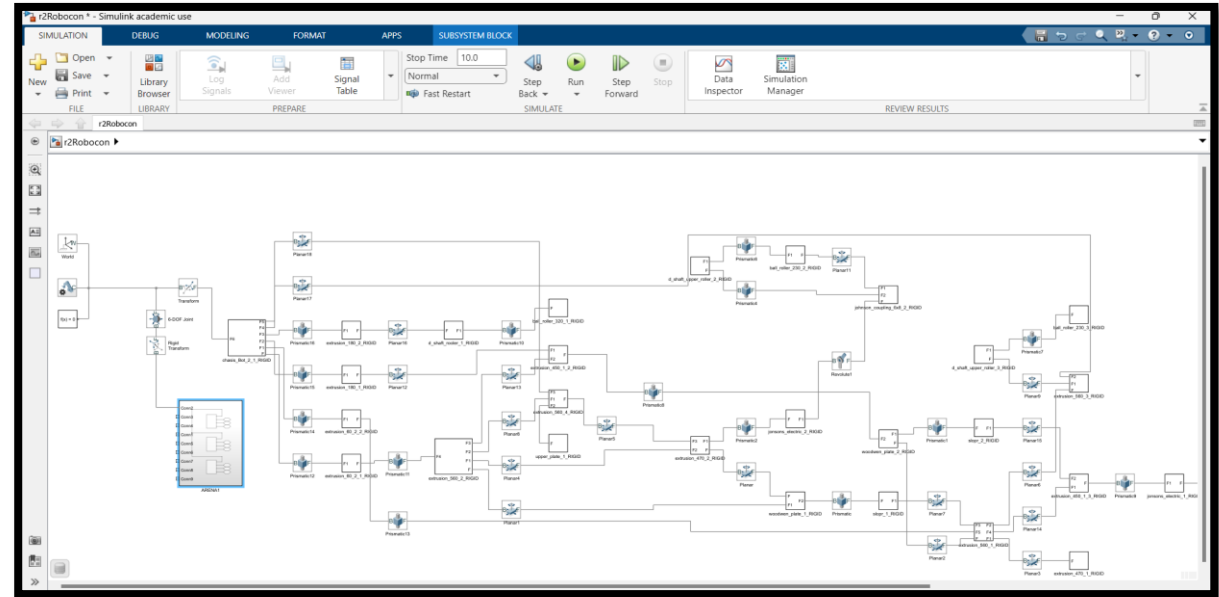


Fig.6. Simscape Multibody of Robot 2

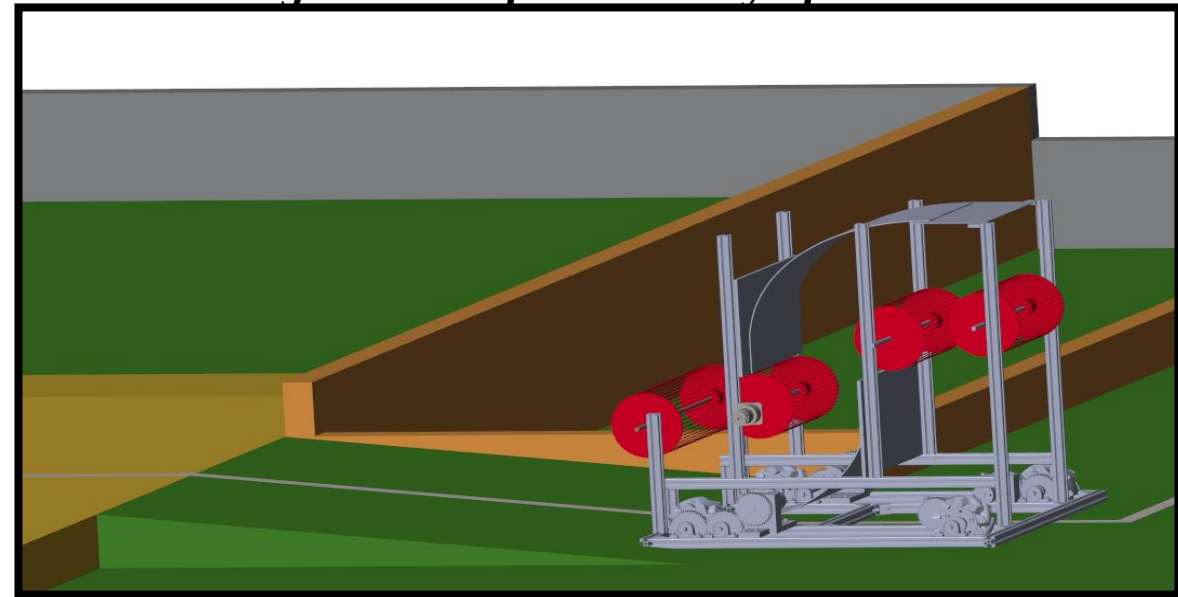


Fig.7. Robot 2 on Arena

Navigation and Path Planning

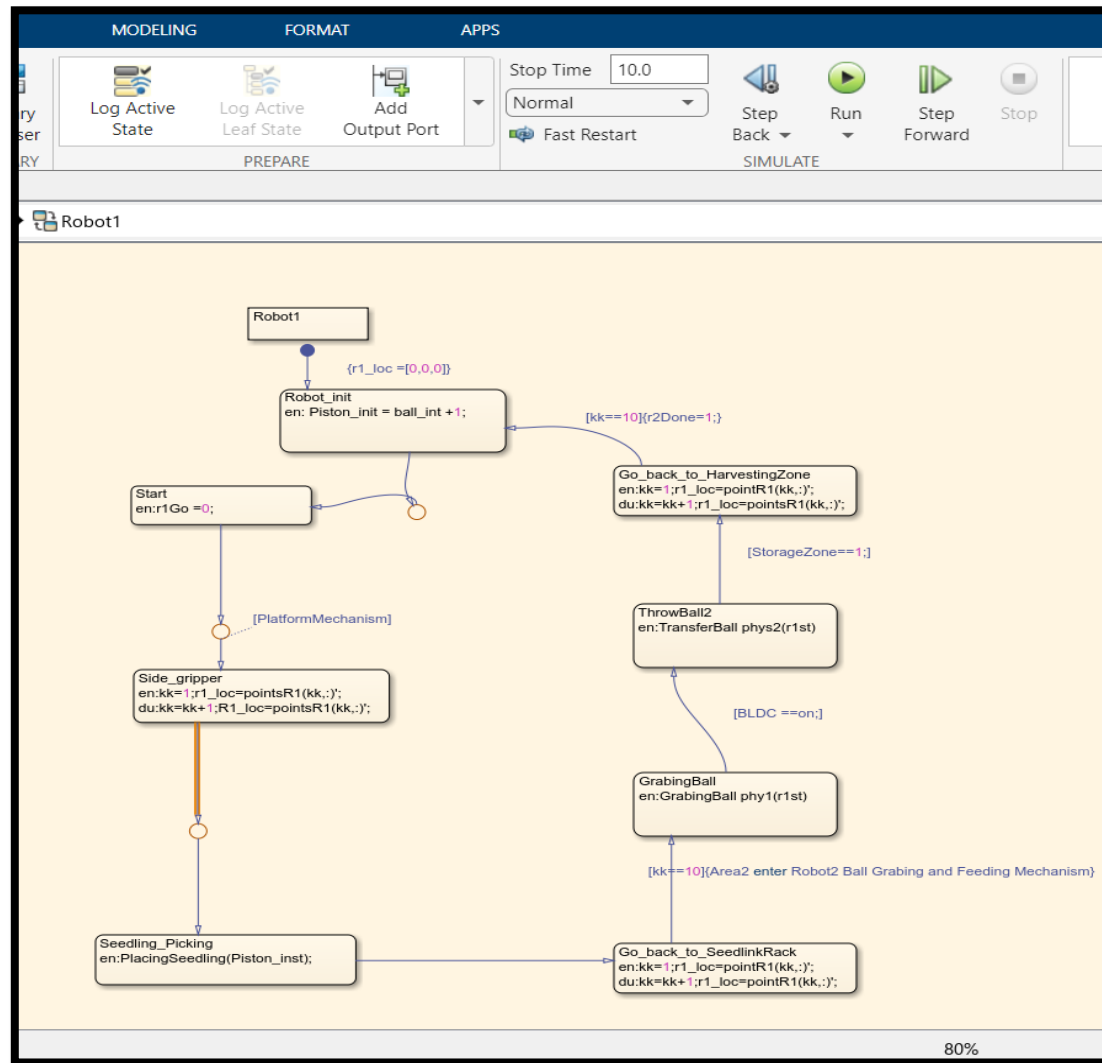
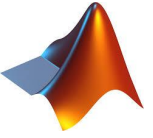


Fig.8. Stateflow of Robot 1

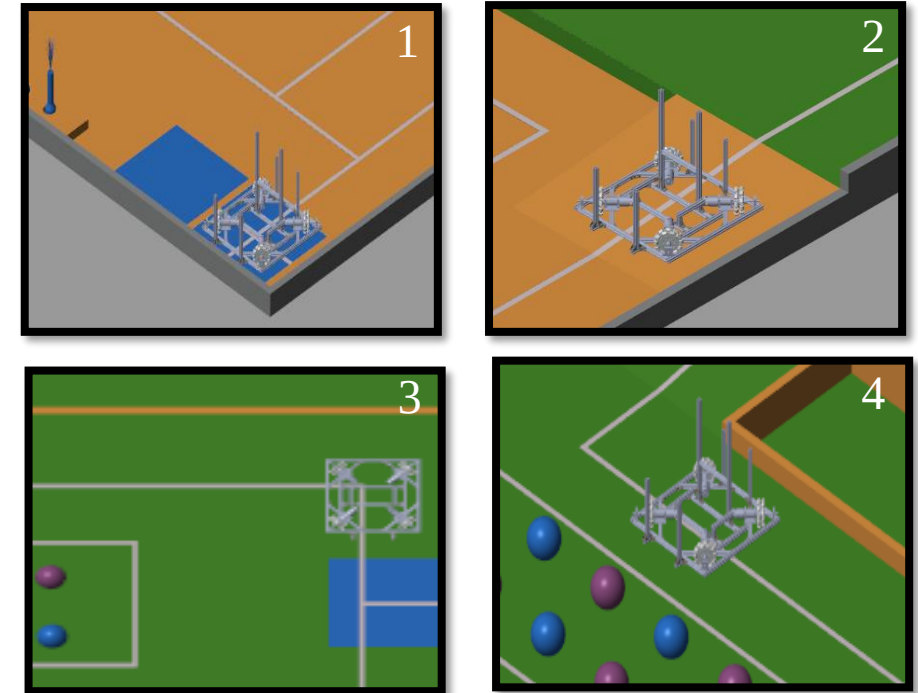


Fig.9. Navigation of holonomic drive of Robot 1

- Stateflow is used to design the algorithm for navigation of Robot 1 and Robot 2 .
- Using Stateflow in MATLAB for robot navigation allowed us to model complex state-based behavior and integrate various control logics effectively.

Stateflow Chart:

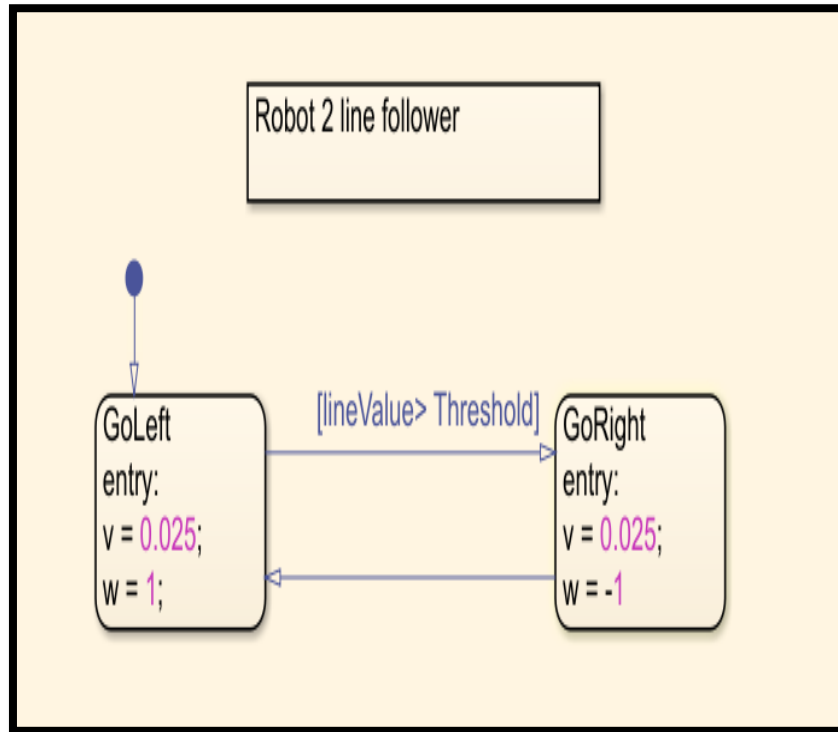


Fig.10(a) Stateflow of Line follower

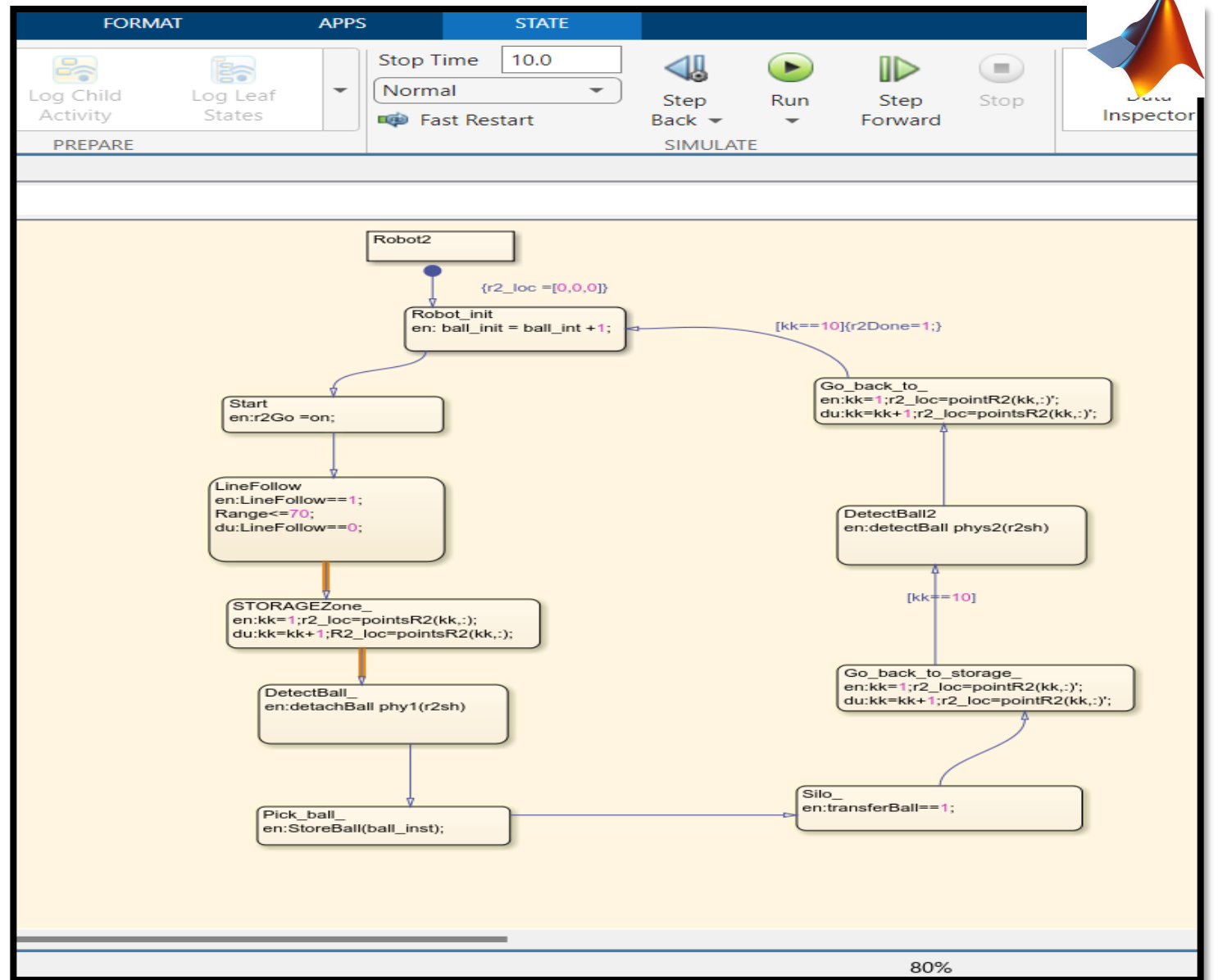
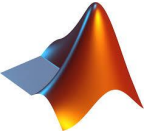


Fig.10(b) Stateflow of Robot 2



State Flow Chart:

- This Stateflow chart (*fig.10(b)*) represents the logic underlying the behavior of Robot 2. The chart contains two primary states: "on" and "off".
- Configurable LSA08 sensor for line following, path following, color detection, and ball detection simulations.
- Visualization of the robot and its sensors using occupancy maps and signal plot graphs.

The **Mode Logic Chart** monitors and controls the status of Robot 2's behavior and movements. The operational sequence for Robot 2 is as follows:

- **Line Following:** Robot 2 uses the LSA08 line follower sensor to navigate along a predefined path.

https://drive.google.com/file/d/1_bFih3e6OiClFV5cki0cJgqX3yMPyRNY/view?usp=drive_link(Refer drive link)

- **Ball Color Detection:** The robot employs a webcam to detect balls based on their color.
- **Ball Grabbing:** After detecting the ball, Robot 2 engages in grabbing the ball.
- **Ball Storage:** Finally, Robot 2 transfers the ball to the designated silo (Silo zone).

Line Follower of Robot 2:

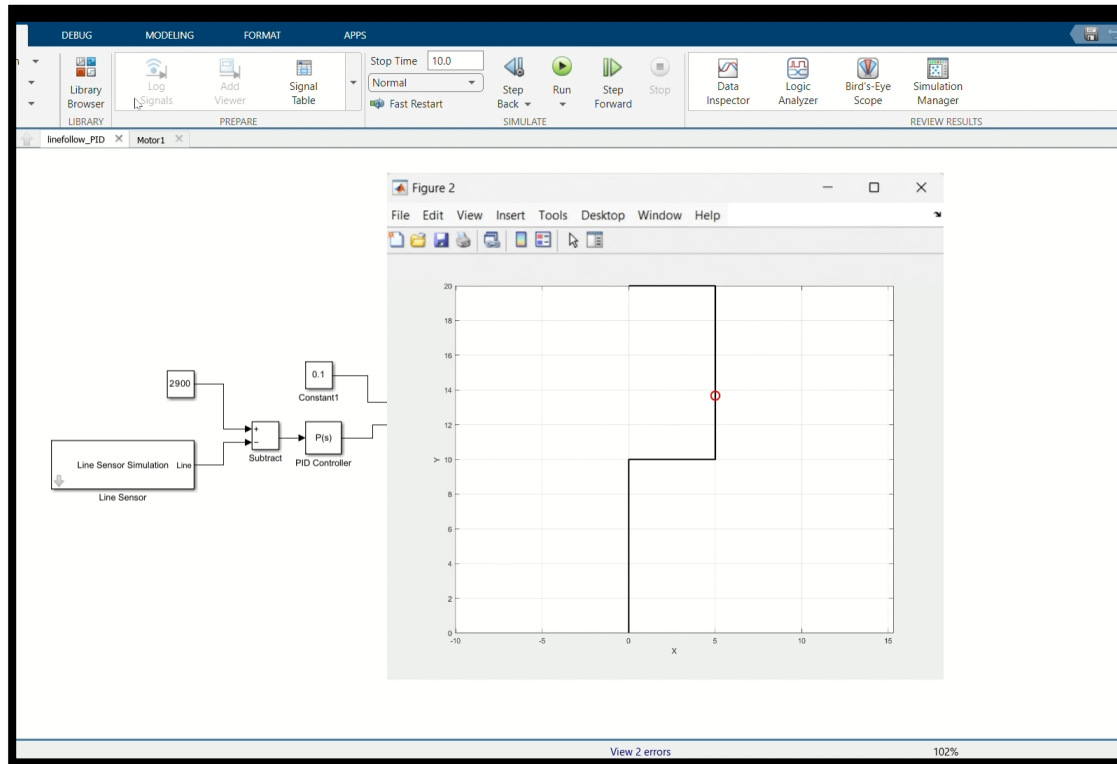
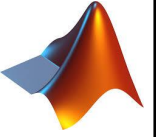


Fig.11. Robot 2 following line

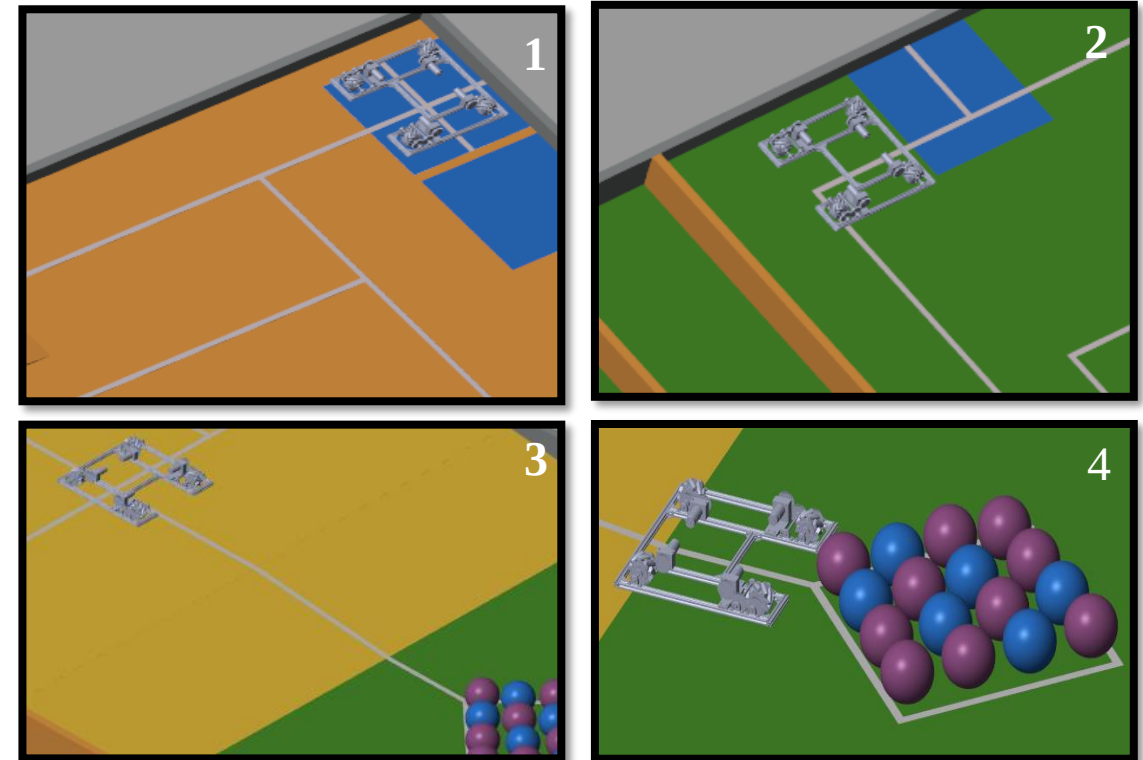


Fig.12. Navigation of holonomic drive of Robot 2

- **Robot Simulator:** We utilized a robot simulator to conduct accurate path simulations. By analyzing the simulated paths and plotting the results, we gain valuable insights for further refinement and optimization of our control algorithms.
- We also used PID controller for designing line following algorithm for Robot 2 using Mobile Robotics Library.

Control Design and PID Tunning

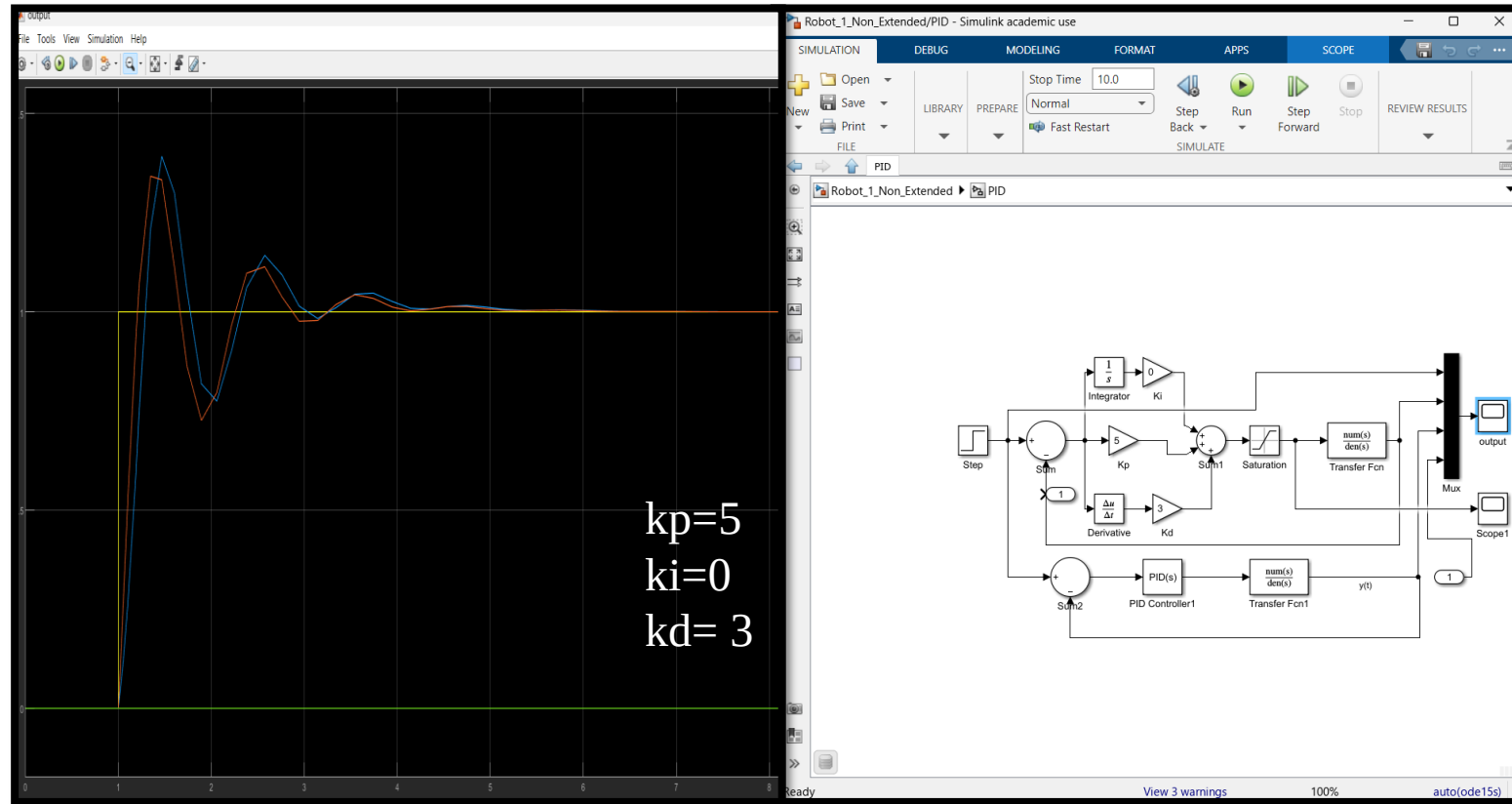
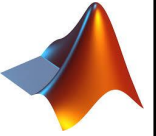


Fig.13. PID for Robot 1

- Designing the control system for locomotion of Robot 1 through its game field.
- We used a PID controllers to control the chassis of the robot. Parameters are tuned to achieve a robust design with desired response. The response is shown in *fig.13*.

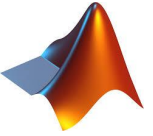


Image Processing:

Color Detection of Paddy Rice: Process Overview:

- **Masking the Object Using Color Thresholder:**
 - Use MATLAB's Color Thresholder app to create and export a mask function based on the color of the paddy rice.
- **Image Calibration:**
 - Calibrate the captured image to correct for lens distortions and align the image accurately.
- **Video Recording:**
 - Initialize the webcam using the webcam() function.
 - Continuously capture snapshots from the webcam for real-time processing.
- **Real-Time Processing:**
 - Apply the mask function to each captured frame to isolate paddy rice.
 - Compare the function's data set with the real-time camera feed.

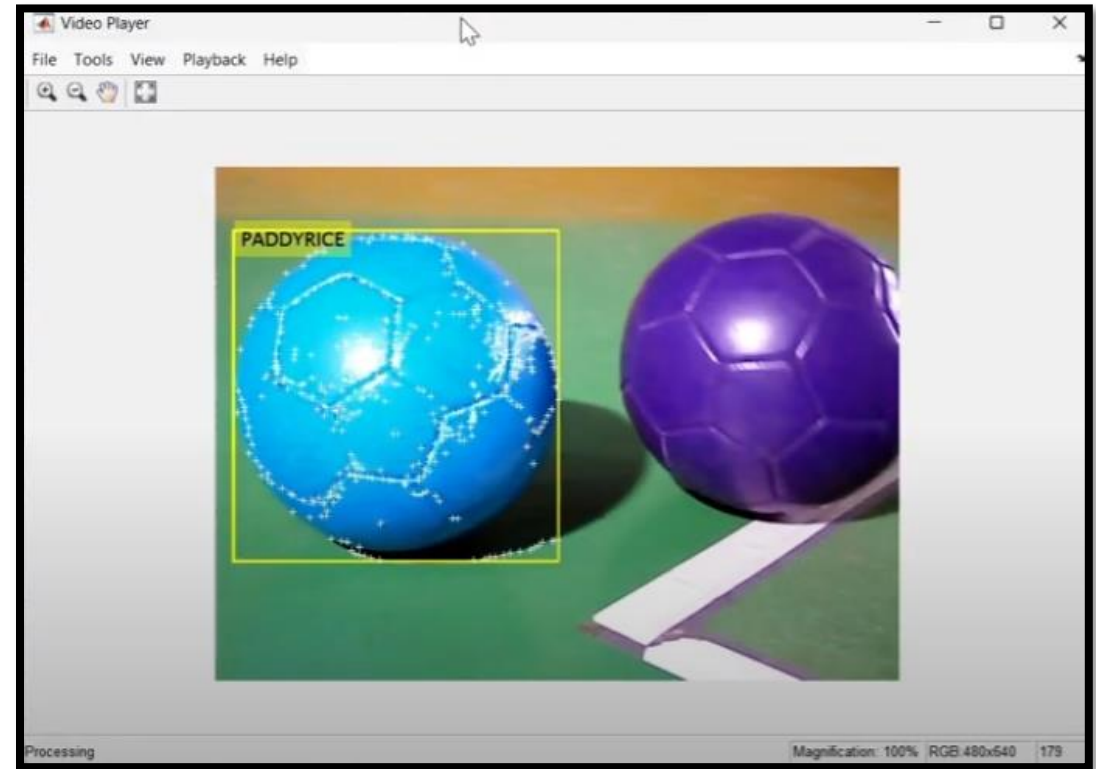
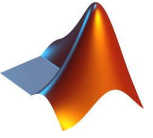


Fig.14. Colour detection

<https://drive.google.com/file/d/14jFmwkbQbEZeYW-TkIk7iv01I3brJSR9/view?usp=sharing>

(Refer above drive link)



Object Detection:

For Object Detection of Paddy Rice following process is followed:

- **Color-Based Object Masking Using Color Thresholder:**
 - Use MATLAB's Color Thresholder app to create and export a color-based mask function for paddy rice.
- **Video Recording Using Webcam:**
 - Initialize the webcam with the webcam() function. Continuously capture snapshots for real-time processing.
- **Object Detection and Tracking:**
 - Apply the mask to each frame to isolate paddy rice. Detect and track objects, calculating centroids and movements.
- **Plotting and Visualization:**
 - Plotted detected objects and their movements on video frames displaying the results in a *fig.15*.

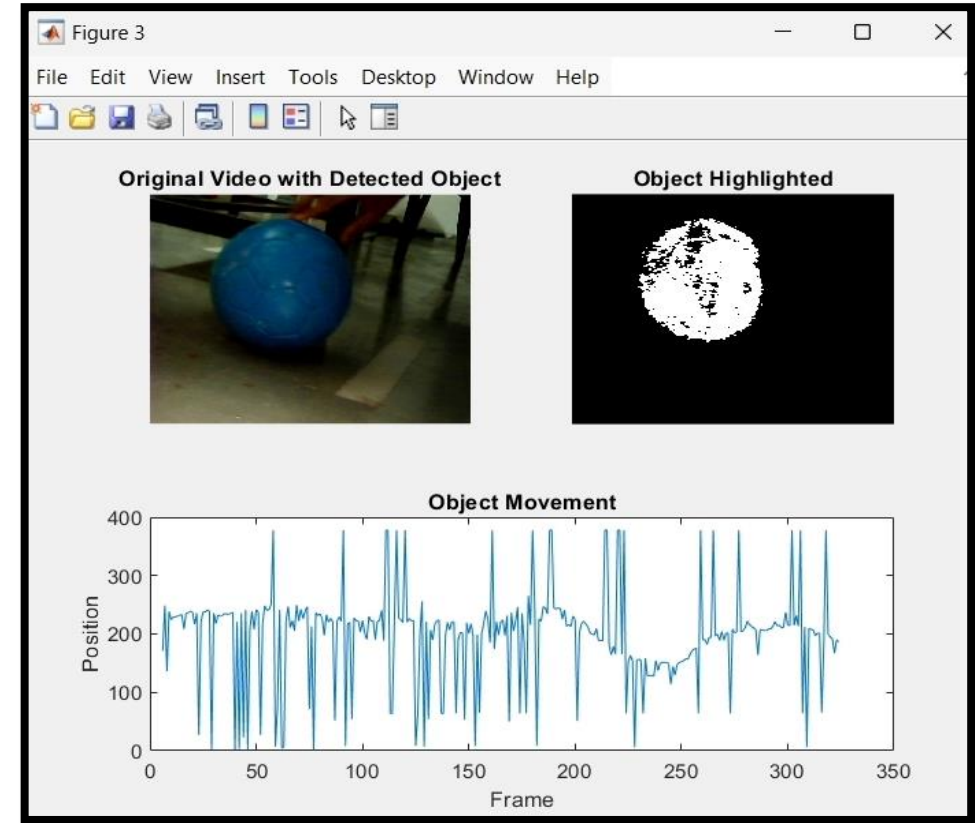


Fig.15. Ball tracking moment using object detection

https://drive.google.com/file/d/1uXwbSr2Ak2Y6vyApZeMfx51oHDyJrsI4/view?usp=drive_link

(Refer above drive link)

Data Analysis

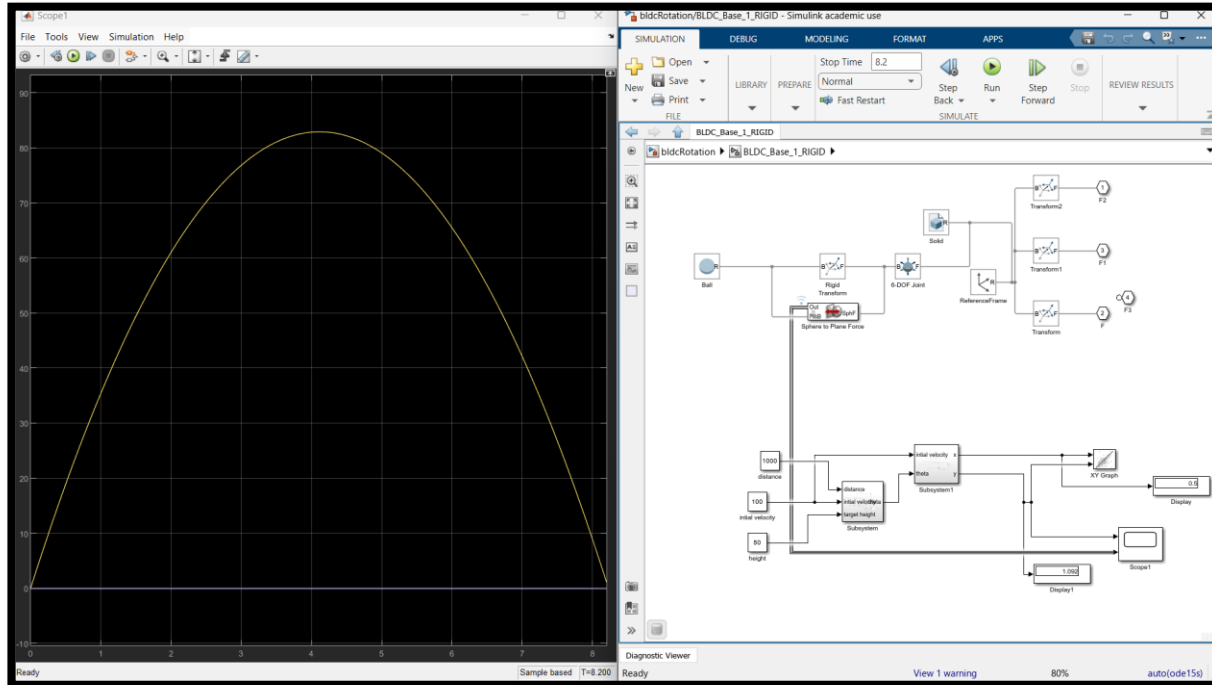
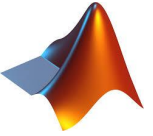


Fig.16. Ball trajectory

Data Analysis of ball trajectory:

➤ Data Acquisition:

- Acquire real-time trajectory data from the BLDC.

➤ Data Analysis:

- Analyze the BLDC transfer mechanism data using the scope shown in *fig.16*.

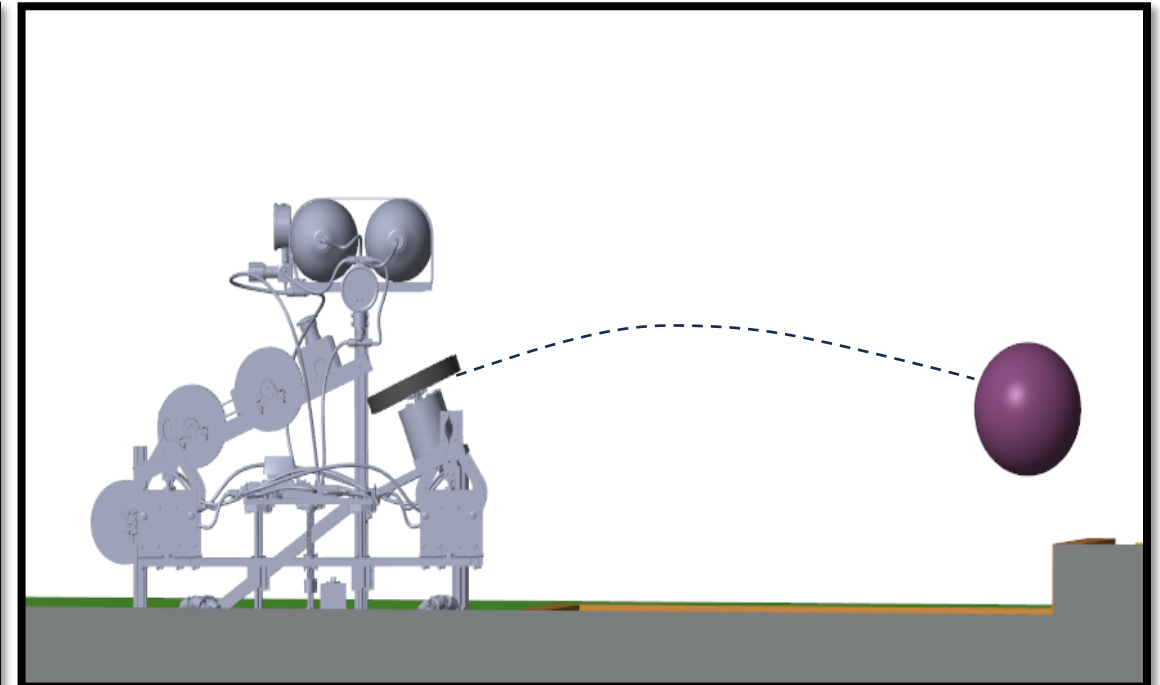


Fig.17. Ball transfer mechanism

➤ Trajectory Analysis:

- Determine the trajectory angle between the BLDC and the storage zone.

https://drive.google.com/file/d/1abkQ63c9_DTKoCeasEPvsN1g075Npfpo/view?usp=drive_link

(Refer above drive link)

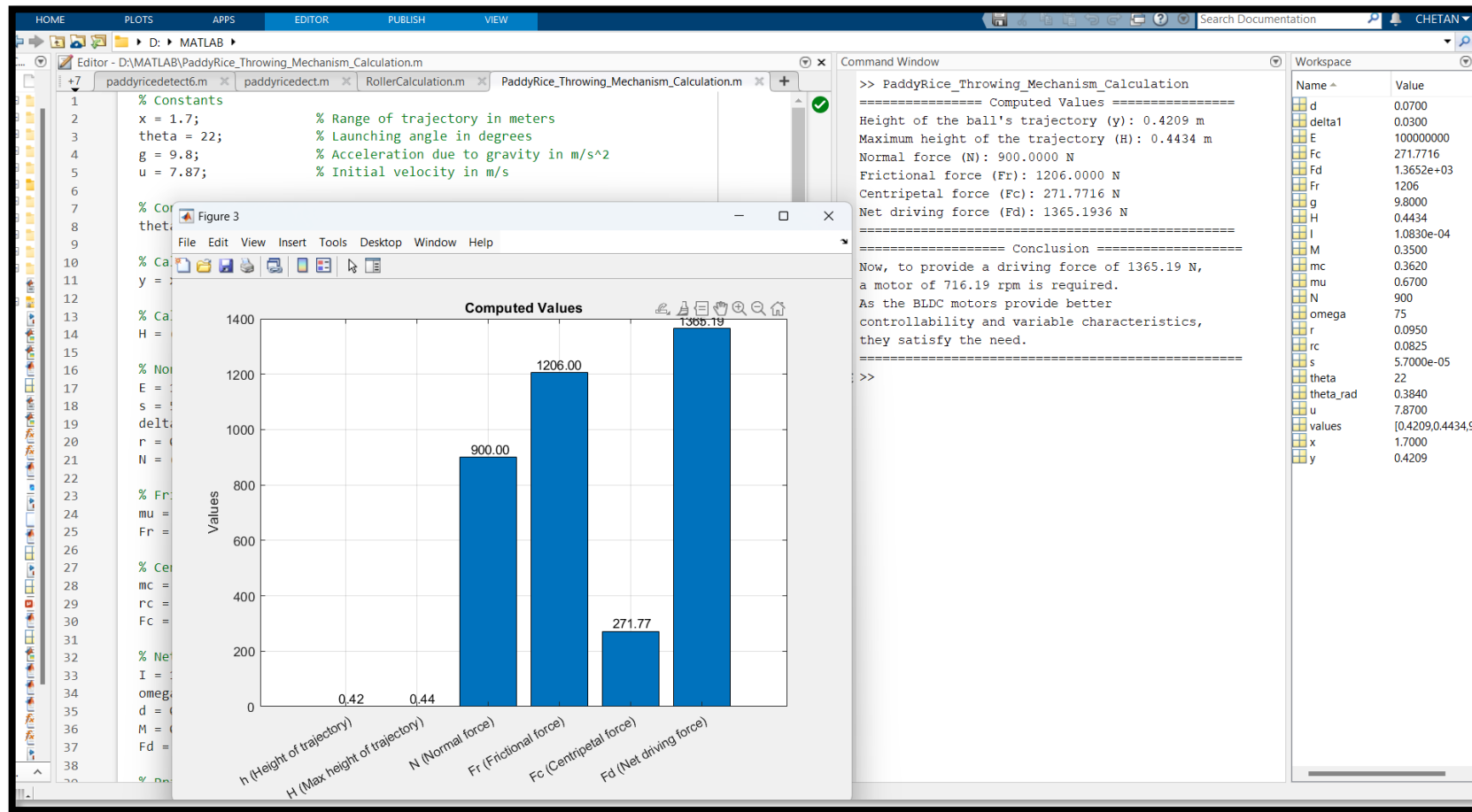


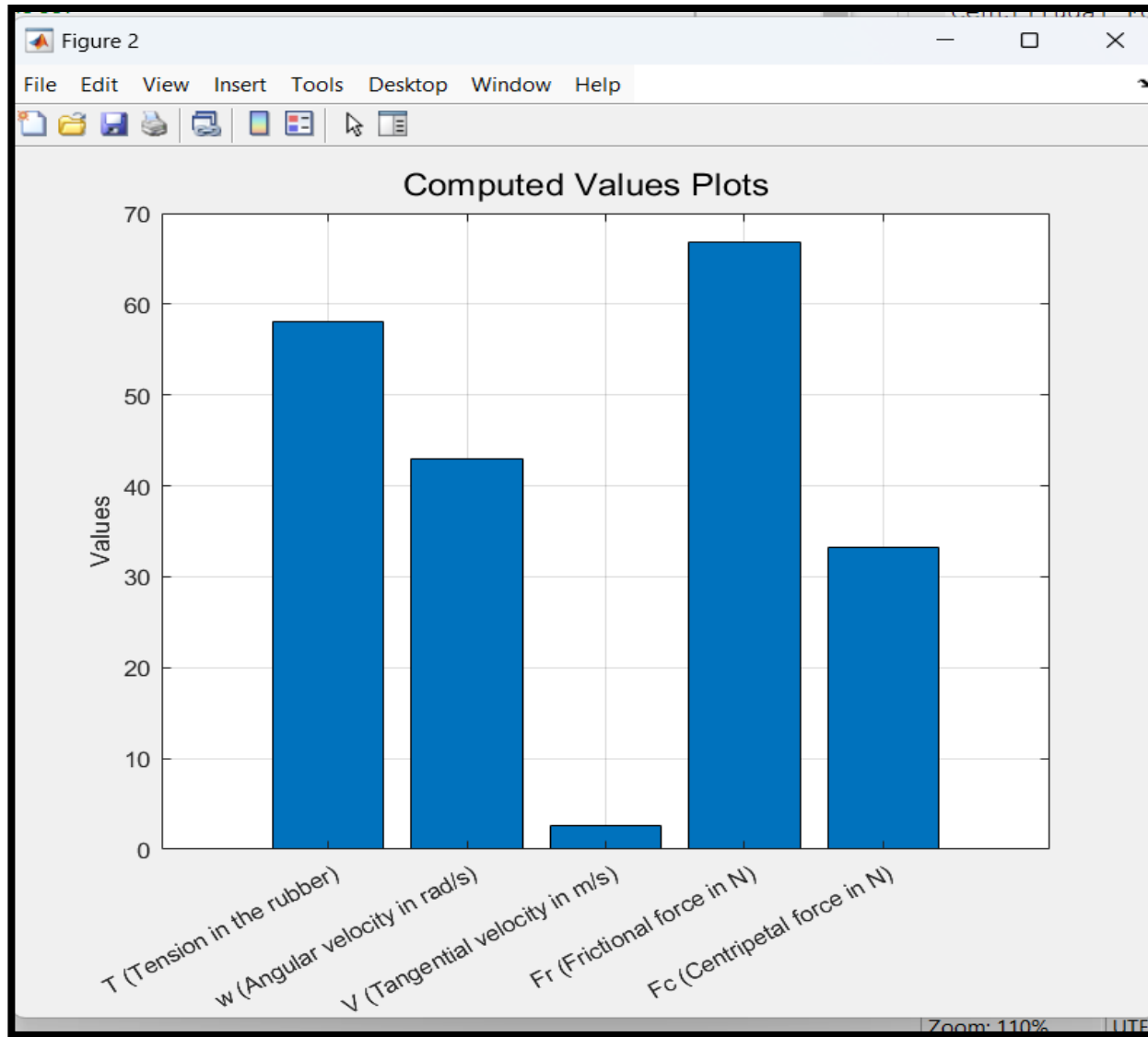
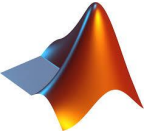
Fig.18. Forces acting of the ball

➤ Initial Conditions:

- Initial velocity: 100 mm/s.
- Contact stiffness: $1e-5$ (N/m).
- Consider acceleration due to gravity and linear force.

➤ Trajectory Characteristics:

- Calculated the maximum height of the ball in the trajectory.
- Plotted the frictional force and normal force as shown in the given *fig.18*.



Data Analysis of Roller Mechanism:

- 1. Roller Mechanism of Robot 1:** It's function is of ball grabbing and feeding to BLDC.
- 2.Materials:** The roller plates are made of acrylic. The tensile strength of the latex rubber is $T=58.03$
- 3.Graphical Representation:** Refer to the provided *fig.19*. for the graphical analysis.

Fig.19. Graph of forces acting on the ball by the Rollers

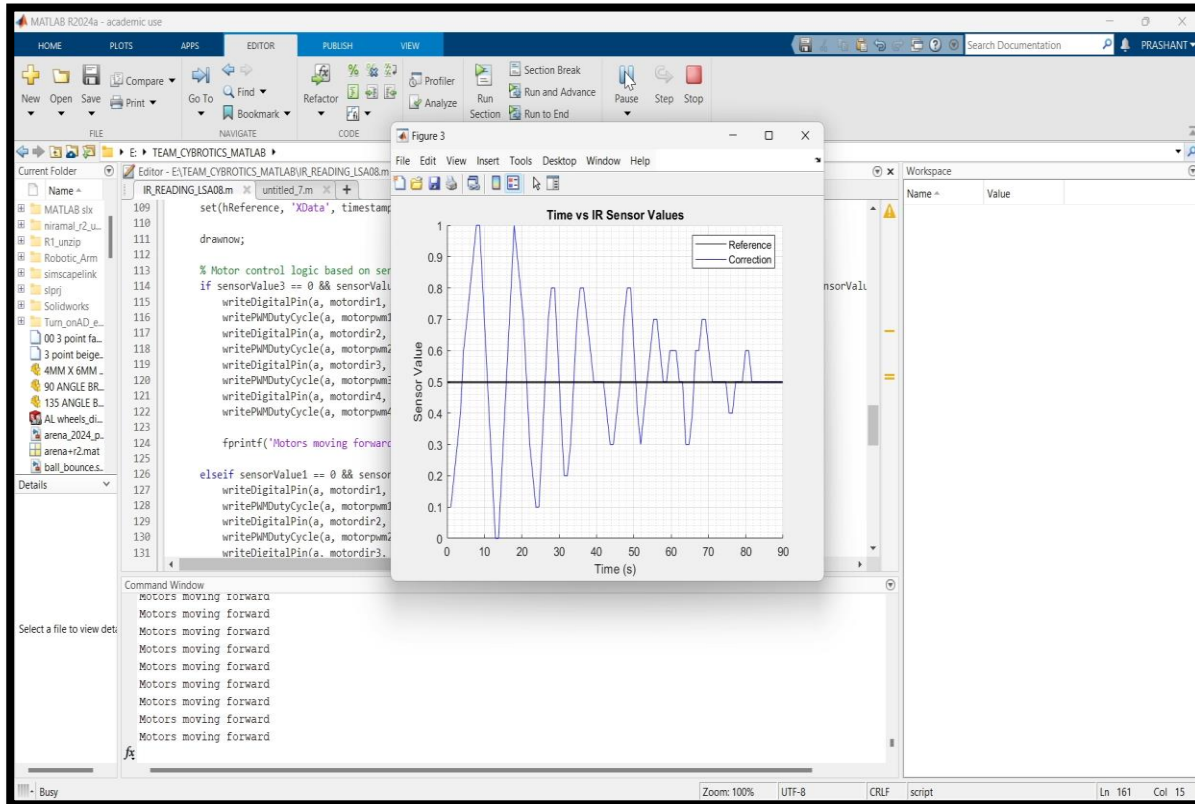
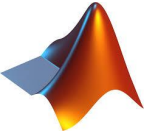


Fig.20. Line following Analysis

Data Analysis of LSA08(Line Follower):

- **LSA08 IR Sensor Array:** Used for line following in the robot.
- **Sensor Configuration:** Utilized the middle two IR sensors, numbered 4 and 5, as reference points.
- ❖ **Negative and Positive Values:**
 - **Sensors before sensor 4:** Detected line to the left, providing negative values.
 - **Sensors after sensor 5:** Detected line to the right, providing positive values.
- **PID Controller:**

Implemented to correct sensor readings and adjust robot movements. Operates based on error signal, which is the disparity between desired and actual positions.
- **Graph Representation:**

Fig.20. depicts how PID controller adjusts robot's movements over time to maintain line following accuracy. Illustrates changes in error signal and PID controller's response for minimizing error.

Embedded Systems

MATLAB Interface with Arduino for Robot Control

- **Embedded Coder Efficiency:** Utilizing Embedded Coder, we developed a highly efficient library capable of swiftly controlling the positions of line sensors and executing PID algorithms on the Robot 2 and the up and down position of platform for seedling picking and placing operation by Robot 1.

https://drive.google.com/file/d/1A3_LG056VYP3ul3rz8VW5o950zIKS_Cw/view?usp=drive_link (Refer drive link)

- **Direct Conversion to C Code:** By directly converting our algorithms into C code, we simplified implementation on hardware, ensuring seamless integration and rapid execution.
- **MATLAB Coder Integration:** MATLAB Coder seamlessly interfaces with Arduino hardware systems, facilitating realistic simulations of actual robot behavior. This integration streamlines the process of developing and deploying control algorithms.
- **Real-Time Execution:** The direct interface between Arduino and MATLAB's Live Editor enables real-time execution of path following algorithms. This setup allows for immediate testing and validation of control strategies.

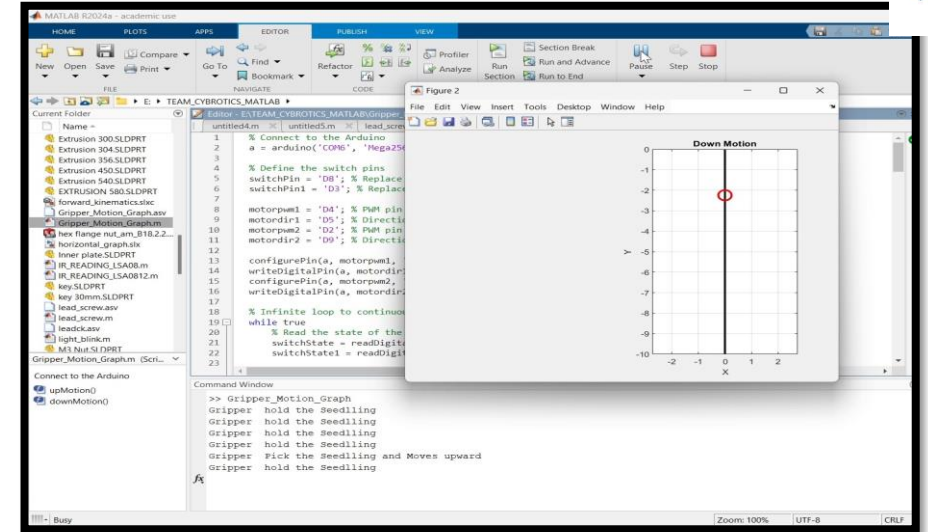


Fig.21. Gripper up motion

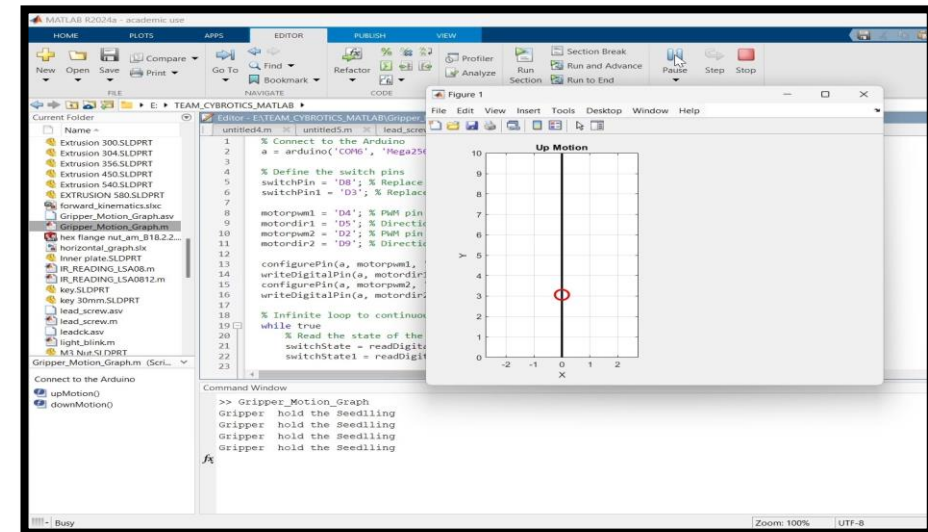
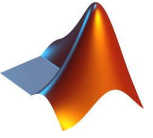


Fig.22. Gripper down motion



Lessons Learnt:

- Learned how to simulate models in MATLAB Simulink.
- Imported CAD models into MATLAB Simulink and generated multibody models using Simscape Multibody.
- Studied body motions, constraints, and rigid transform blocks for rotational and translational motions.
- Explored image processing, mapping, color detection, and programming.
- Analyzed data and signals using graphs.
- Worked on real-time simulation, problem-solving, and debugging simulation configurations.
- Investigated the actual behavior of robots and mechanical components in Simscape Multibody using joints and constraints.
- Utilized various Simulink libraries and tools.

Future Plans:

- We will improve the algorithms using Embedded design.
- We will implement the modeling and simulation to make a more realistic design.
- Use computer vision for processing algorithms and avoid the complexity of sensors.
- Design efficient control systems for stability, safety, and reliability.

Missing from the tools:

- Execution time of tools.
- Requirement of high processing unit.

...THANK YOU!